Matrix Cheatsheet Table

[back to article]

(last updated: June 22, 2018)

CREATING MATRICES

Creating Matrices

(here: 3x3 matrix)

MATLAB/Octave
```
M> A = [1 2 3; 4 5 6; 7 8 9]

A =
   1   2   3
   4   5   6
   7   8   9
```

Python NumPy
```
P> A = np.array([ [1,2,3], [4,5,6],
[7,8,9] ])

P> A
array([[1, 2, 3],
       [4, 5, 6],
       [7, 8, 9]])
```

R
```
R> A =
matrix(c(1,2,3,4,5,6,7,8,9),nrow=3,byr

 # equivalent to
# A = matrix(1:9,nrow=3,byrow=T)

R> A
[,1] [,2] [,3]
[1,] 1 2 3
[2,] 4 5 6
[3,] 7 8 9
```

Creating an column vector (nx1 matrix)

MATLAB/Octave
```
M> a = [1; 2; 3]
a =

   1

   2

   3
```

Python NumPy
```
P> a = np.array([1,2,3]).reshape(3,1)



P> b.shape
(3, 1)
```

R
```
R> a = matrix(c(1,2,3), nrow=3, byrow

R> a
[,1]
[1,] 1
[2,] 2
[3,] 3
```

Creating an row vector (1xn matrix)

MATLAB/Octave
```
M> b = [1 2 3]
b =
   1   2   3
```

Python NumPy
```
P> b = np.array([1,2,3]).reshape(1, 3)

P> b
array([[1],
       [2],
       [3]])

# note that in numpy, 1D arrays
# can be multiplied
# with 2d arrays, too



P> b.shape

(1, 3)
```

R
```
R> b = matrix(c(1,2,3), ncol=3)

R> b
[,1] [,2] [,3]

[1,] 1 2 3
```

Creating a random m x n matrix

MATLAB/Octave
```
M> rand(3,2)

ans =

   0.21977   0.10220

   0.38959   0.69911

   0.15624   0.65637
```

Python NumPy
```
P> np.random.rand(3,2)

array([[ 0.29347865,  0.17920462],

       [ 0.51615758,  0.64593471],

       [ 0.01067605,  0.09692771]])
```

R
```
R> matrix(runif(3*2), ncol=2)

[,1] [,2]

[1,] 0.5675127 0.7751204

[2,] 0.3439412 0.5261893

[3,] 0.2273177 0.223438
```

Creating a zero m x n matrix

MATLAB/Octave
```
M> zeros(3,2)

ans =
```

Python NumPy
```
P> np.zeros((3,2))

array([[ 0.,  0.],
```

R
```
R> mat.or.vec(3, 2)

[,1] [,2]
```

	0 0 0 0 0 0	[0., 0.], [0., 0.]])	[1,] 0 0 [2,] 0 0 [3,] 0 0
Creating an m x n matrix of ones	M> ones(3,2) ans = 1 1 1 1 1 1	P> np.ones((3,2)) array([[1., 1.], [1., 1.], [1., 1.]])	R> matrix(1L, 3, 2) [,1] [,2] [1,] 1 1 [2,] 1 1 [3,] 1 1
Creating an identity matrix	M> eye(3) ans = Diagonal Matrix 1 0 0 0 1 0 0 0 1	P> np.eye(3) array([[1., 0., 0.], [0., 1., 0.], [0., 0., 1.]])	R> diag(3) [,1] [,2] [,3] [1,] 1 0 0 [2,] 0 1 0 [3,] 0 0 1
Creating a diagonal matrix	M> a = [1 2 3] M> diag(a) ans = Diagonal Matrix 1 0 0 0 2 0 0 0 3	P> a = np.array([1,2,3]) P> np.diag(a) array([[1, 0, 0], [0, 2, 0], [0, 0, 3]])	R> diag(1:3) [,1] [,2] [,3] [1,] 1 0 0 [2,] 0 2 0 [3,] 0 0 3

ACCESSING MATRIX ELEMENTS

Getting the dimension of a matrix (here: 2D, rows x cols)	M> A = [1 2 3; 4 5 6] A = 1 2 3 4 5 6 M> size(A) ans = 2 3	P> A = np.array([[1,2,3], [4,5,6]]) P> A array([[1, 2, 3], [4, 5, 6]]) P> A.shape (2, 3)	R> A = matrix(1:6,nrow=2,byrow=T) R> A [,1] [,2] [,3] [1,] 1 2 3 [2,] 4 5 6 R> dim(A) [1] 2 3
Selecting rows	M> A = [1 2 3; 4 5 6; 7 8 9] % 1st row M> A(1,:) ans = 1 2 3 % 1st 2 rows	P> A = np.array([[1,2,3], [4,5,6], [7,8,9]]) # 1st row P> A[0,:] array([1, 2, 3]) # 1st 2 rows P> A[0:2,:]	R> A = matrix(1:9,nrow=3,byrow=T) # 1st row R> A[1,] [1] 1 2 3 # 1st 2 rows R> A[1:2,]

	M> A(1:2,:) ans = 1 2 3 4 5 6	array([[1, 2, 3], [4, 5, 6]])	[,1] [,2] [,3] [1,] 1 2 3 [2,] 4 5 6
Selecting columns	M> A = [1 2 3; 4 5 6; 7 8 9] % 1st column M> A(:,1) ans = 1 4 7 % 1st 2 columns M> A(:,1:2) ans = 1 2 4 5 7 8	P> A = np.array([[1,2,3], [4,5,6], [7,8,9]]) # 1st column (as row vector) P> A[:,0] array([1, 4, 7]) # 1st column (as column vector) P> A[:,[0]] array([[1], [4], [7]]) # 1st 2 columns P> A[:,0:2] array([[1, 2], [4, 5], [7, 8]])	R> A = matrix(1:9,nrow=3,byrow=T) # 1st column as row vector R> t(A[,1]) [,1] [,2] [,3] [1,] 1 4 7 # 1st column as column vector R> A[,1] [1] 1 4 7 # 1st 2 columns R> A[,1:2] [,1] [,2] [1,] 1 2 [2,] 4 5 [3,] 7 8
Extracting rows and columns by criteria **(here: get rows that have value 9 in column 3)**	M> A = [1 2 3; 4 5 9; 7 8 9] A = 1 2 3 4 5 9 7 8 9 M> A(A(:,3) == 9,:) ans = 4 5 9 7 8 9	P> A = np.array([[1,2,3], [4,5,9], [7,8,9]]) P> A array([[1, 2, 3], [4, 5, 9], [7, 8, 9]]) P> A[A[:,2] == 9] array([[4, 5, 9], [7, 8, 9]])	R> A = matrix(1:9,nrow=3,byrow=T) R> A [,1] [,2] [,3] [1,] 1 2 3 [2,] 4 5 9 [3,] 7 8 9 R> A[A[,3]==9,] [1] 7 8 9
Accessing elements **(here: 1st element)**	M> A = [1 2 3; 4 5 6; 7 8 9] M> A(1,1) ans = 1	P> A = np.array([[1,2,3], [4,5,6], [7,8,9]]) P> A[0,0] 1	R> A = matrix(c(1,2,3,4,5,9,7,8,9),nrow=3,byr R> A[1,1] [1] 1

MANIPULATING SHAPE AND DIMENSIONS

Converting **a matrix into a row vector (by column)**	M> A = [1 2 3; 4 5 6; 7 8 9] M> A(:)	P> A = np.array([[1,2,3],[4,5,6],[7,8,9]]) P> A.flatten(1) # returns a copy	R> A = matrix(1:9,nrow=3,byrow=T) R> as.vector(A)

	ans =	array([1, 4, 7, 2, 5, 8, 3, 6, 9])	
		1# alternatively A.ravel()	[1] 1 4 7 2 5 8 3 6 9
		4# ravel() returns a view	
		7	
		2	
		5	
		8	
		3	
		6	
		9	
Converting row to column vectors	M> b = [1 2 3]	P> b = np.array([1, 2, 3])	R> b = matrix(c(1,2,3), ncol=3)
	M> b = b'	P> b = b[np.newaxis].T	R> t(b)
	b =	# alternatively	[,1]
	1	# b = b[:,np.newaxis]	[1,] 1
	2		[2,] 2
	3	P> b	[3,] 3
		array([[1],	
		[2],	
		[3]])	
Reshaping Matrices	M> A = [1 2 3; 4 5 6; 7 8 9]	P> A = np.array([[1,2,3],[4,5,6],[7,8,9]])	R> A = matrix(1:9,nrow=3,byrow=T)
	A =		
(here: 3x3 matrix to row vector)	1 2 3	P> A	R> A
	4 5 6	array([[1, 2, 3],	[,1] [,2] [,3]
	7 8 9	[4, 5, 9],	[1,] 1 2 3
		[7, 8, 9]])	[2,] 4 5 6
	M> total_elements = numel(A)		[3,] 7 8 9
		P> total_elements = np.prod(A.shape)	
	M> B = reshape(A,1,total_elements)		R> total_elements = dim(A)[1] * dim(A
	% or reshape(A,1,9)	P> B = A.reshape(1, total_elements)	
	B =		R> B = matrix(A, ncol=total_elements)
	1 4 7 2 5 8 3 6 9	# alternative shortcut:	
		# A.reshape(1,-1)	R> B
			[,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8
		P> B	[1,] 1 4 7 2 5 8 3 6 9
		array([[1, 2, 3, 4, 5, 6, 7, 8, 9]])	
Concatenating matrices	M> A = [1 2 3; 4 5 6]	P> A = np.array([[1, 2, 3], [4, 5, 6]])	R> A = matrix(1:6,nrow=2,byrow=T)
	M> B = [7 8 9; 10 11 12]	P> B = np.array([[7, 8, 9],[10,11,12]])	R> B = matrix(7:12,nrow=2,byrow=T)
	M> C = [A; B]	P> C = np.concatenate((A, B), axis=0)	R> C = rbind(A,B)
	1 2 3		
	4 5 6	P> C	R> C
	7 8 9	array([[1, 2, 3],	[,1] [,2] [,3]
	10 11 12	[4, 5, 6],	[1,] 1 2 3
		[7, 8, 9],	[2,] 4 5 6
		[10, 11, 12]])	[3,] 7 8 9
			[4,] 10 11 12
Stacking vectors and matrices	M> a = [1 2 3]	P> a = np.array([1,2,3])	R> a = matrix(1:3, ncol=3)
		P> b = np.array([4,5,6])	
	M> b = [4 5 6]		R> b = matrix(4:6, ncol=3)
		P> np.column_stack([a,b])	
	M> c = [a' b']	array([[1, 4],	R> matrix(rbind(A, B), ncol=2)
	c =	[2, 5],	[,1] [,2]
	1 4	[3, 6]])	[1,] 1 5

```
   2   5
   3   6

M> c = [a; b]
c =
   1   2   3
   4   5   6
```

```
P> np.row_stack([a,b])
array([[1, 2, 3],
       [4, 5, 6]])
```

```
[2,] 4 3

 R> rbind(A,B)
[,1] [,2] [,3]
[1,] 1 2 3
[2,] 4 5 6
```

BASIC MATRIX OPERATIONS

Matrix-scalar operations

```
M> A = [1 2 3; 4 5 6; 7 8 9]

M> A * 2
ans =
    2    4    6
    8   10   12
   14   16   18

M> A + 2

M> A - 2

M> A / 2
```

```
P> A = np.array([ [1,2,3], [4,5,6], [7,8,9] ])

P> A * 2
array([[ 2,  4,  6],
       [ 8, 10, 12],
       [14, 16, 18]])

P> A + 2

P> A - 2

P> A / 2

# Note that NumPy was optimized for
# in-place assignments
# e.g., A += A instead of
# A = A + A
```

```
R> A = matrix(1:9, nrow=3, byrow=T)

R> A * 2
[,1] [,2] [,3]
[1,] 2 4 6
[2,] 8 10 12
[3,] 14 16 18

R> A + 2

R> A - 2

R> A / 2
```

Matrix-matrix multiplication

```
M> A = [1 2 3; 4 5 6; 7 8 9]

M> A * A
ans =
    30    36    42
    66    81    96
   102   126   150
```

```
P> A = np.array([ [1,2,3], [4,5,6], [7,8,9] ])

P> np.dot(A,A) # or A.dot(A)
array([[ 30,  36,  42],
       [ 66,  81,  96],
       [102, 126, 150]])
```

```
R> A = matrix(1:9, nrow=3, byrow=T)

R> A %*% A
[,1] [,2] [,3]
[1,] 30 36 42
[2,] 66 81 96
[3,] 102 126 150
```

Matrix-vector multiplication

```
M> A = [1 2 3; 4 5 6; 7 8 9]

M> b = [ 1; 2; 3 ]

M> A * b
ans =
   14
   32
   50
```

```
P> A = np.array([ [1,2,3], [4,5,6], [7,8,9] ])

P> b = np.array([ [1], [2], [3] ])

P> np.dot(A,b) # or A.dot(b)

array([[14], [32], [50]])
```

```
R> A = matrix(1:9, ncol=3)

R> b = matrix(1:3, nrow=3)

 R> t(b %*% A)
[,1]
[1,] 14
[2,] 32
[3,] 50
```

Element-wise matrix-matrix operations

```
M> A = [1 2 3; 4 5 6; 7 8 9]

M> A .* A
ans =
    1    4    9
   16   25   36
```

```
P> A = np.array([ [1,2,3], [4,5,6], [7,8,9] ])

P> A * A
array([[ 1,  4,  9],
       [16, 25, 36],
       [49, 64, 81]])
```

```
R> A = matrix(1:9, nrow=3, byrow=T)

R> A * A
[,1] [,2] [,3]
[1,] 1 4 9
[2,] 16 25 36
```

	49 64 81 M> A .+ A M> A .- A M> A ./ A	P> A + A P> A - A P> A / A # Note that NumPy was optimized for # in-place assignments # e.g., A += A instead of # A = A + A	[3,] 49 64 81 R> A + A R> A - A R> A / A
Matrix elements to power n	M> A = [1 2 3; 4 5 6; 7 8 9]	P> A = np.array([[1,2,3], [4,5,6], [7,8,9]])	R> A = matrix(1:9, nrow=3, byrow=T)
(here: individual elements squared)	M> A.^2 ans = 1 4 9 16 25 36 49 64 81	P> np.power(A,2) array([[1, 4, 9], [16, 25, 36], [49, 64, 81]])	R> A ^ 2 [,1] [,2] [,3] [1,] 1 4 9 [2,] 16 25 36 [3,] 49 64 81
Matrix to power n	M> A = [1 2 3; 4 5 6; 7 8 9]	P> A = np.array([[1,2,3], [4,5,6], [7,8,9]])	R> A = matrix(1:9, ncol=3)
(here: matrix-matrix multiplication with itself)	M> A ^ 2 ans = 30 36 42 66 81 96 102 126 150	P> np.linalg.matrix_power(A,2) array([[30, 36, 42], [66, 81, 96], [102, 126, 150]])	# requires the expm package R> install.packages('expm') R> library(expm) R> A %^% 2 [,1] [,2] [,3] [1,] 30 66 102 [2,] 36 81 126 [3,] 42 96 150
Matrix transpose	M> A = [1 2 3; 4 5 6; 7 8 9] M> A' ans = 1 4 7 2 5 8 3 6 9	P> A = np.array([[1,2,3], [4,5,6], [7,8,9]]) P> A.T array([[1, 4, 7], [2, 5, 8], [3, 6, 9]])	R> A = matrix(1:9, nrow=3, byrow=T) R> t(A) [,1] [,2] [,3] [1,] 1 4 7 [2,] 2 5 8 [3,] 3 6 9
Determinant of a matrix: **A -> |A|**	M> A = [6 1 1; 4 -2 5; 2 8 7] A = 6 1 1 4 -2 5 2 8 7 M> det(A) ans = -306	P> A = np.array([[6,1,1],[4,-2,5], [2,8,7]]) P> A array([[6, 1, 1], [4, -2, 5], [2, 8, 7]]) P> np.linalg.det(A) -306	R> A = matrix(c(6,1,1,4,-2,5,2,8,7), byrow=T) R> A [,1] [,2] [,3] [1,] 6 1 1 [2,] 4 -2 5 [3,] 2 8 7 R> det(A) [1] -306
Inverse of a matrix	M> A = [4 7; 2 6] A =	P> A = np.array([[4, 7], [2, 6]])	R> A = matrix(c(4,7,2,6), nrow=2, byr

```
   4   7
   2   6


M> A_inv = inv(A)
A_inv =
   0.60000  -0.70000
  -0.20000   0.40000
```

```
P> A
array([[4, 7],
       [2, 6]])

P> A_inverse = np.linalg.inv(A)

P> A_inverse
array([[ 0.6, -0.7],
       [-0.2, 0.4]])
```

```
R> A
[,1] [,2]
[1,] 4 7
[2,] 2 6

R> solve(A)
[,1] [,2]
[1,] 0.6 -0.7
[2,] -0.2 0.4
```

ADVANCED MATRIX OPERATIONS

Calculating the covariance matrix of 3 random variables (here: covariances of the means of x1, x2, and x3)

```
M> x1 = [4.0000 4.2000 3.9000 4.3000 4.1000]

M> x2 = [2.0000 2.1000 2.0000 2.1000 2.2000]'

M> x3 = [0.60000 0.59000 0.58000 0.62000 0.63000]

M> cov( [x1,x2,x3] )

ans =

   2.5000e-02   7.5000e-03   1.7500e-03
   7.5000e-03   7.0000e-03   1.3500e-03
   1.7500e-03   1.3500e-03   4.3000e-04
```

```
P> x1 = np.array([ 4, 4.2, 3.9, 4.3, 4.1])

P> x2 = np.array([ 2, 2.1, 2, 2.1, 2.2])

P> x3 = np.array([ 0.6, 0.59, 0.58, 0.62, 0.63])

P> np.cov([x1, x2, x3])

Array([[ 0.025  ,  0.0075 ,  0.00175],
       [ 0.0075 ,  0.007  ,  0.00135],
       [ 0.00175,  0.00135,  0.00043]])
```

```
R> x1 = matrix(c(4, 4.2, 3.9, 4.3, 4.
ncol=5)

R> x2 = matrix(c(2, 2.1, 2, 2.1, 2.2)

R> x3 = matrix(c(0.6, 0.59, 0.58, 0.6
ncol=5)

R> cov(matrix(c(x1, x2, x3), ncol=3))

[,1] [,2] [,3]
[1,] 0.02500 0.00750 0.00175
[2,] 0.00750 0.00700 0.00135
[3,] 0.00175 0.00135 0.00043
```

Calculating eigenvectors and eigenvalues

```
M> A = [3 1; 1 3]
A =
   3   1
   1   3

M> [eig_vec,eig_val] = eig(A)
eig_vec =
  -0.70711   0.70711
   0.70711   0.70711
eig_val =
Diagonal Matrix
   2   0
   0   4
```

```
P> A = np.array([[3, 1], [1, 3]])

P> A
array([[3, 1],
       [1, 3]])

P> eig_val, eig_vec = np.linalg.eig(A)

P> eig_val
array([ 4.,  2.])

P> eig_vec
Array([[ 0.70710678, -0.70710678],
       [ 0.70710678,  0.70710678]])
```

```
R> A
[,1] [,2]
[1,] 3 1
[2,] 1 3

R> eigen(A)
$values
[1] 4 2

$vectors
[,1] [,2]
[1,] 0.7071068 -0.7071068
[2,] 0.7071068 0.7071068
```

Generating a Gaussian dataset:

creating random vectors from the multivariate normal distribution given mean and covariance matrix

(here: 5 random vectors with mean 0, covariance = 0, variance = 2)

```
% requires statistics toolbox package

% how to install and load it in Octave:

% download the package from:
%
http://octave.sourceforge.net/packages.php
% pkg install
%     ~/Desktop/io-2.0.2.tar.gz
% pkg install
%     ~/Desktop/statistics-1.2.3.tar.gz

M> pkg load statistics

M> mean = [0 0]

M> cov = [2 0; 0 2]
cov =
   2   0
   0   2

M> mvnrnd(mean,cov,5)
   2.480150  -0.559906
  -2.933047   0.560212
   0.098206   3.055316
  -0.985215  -0.990936
   1.122528   0.686977
```

```
P> mean = np.array([0,0])

P> cov = np.array([[2,0],[0,2]])

P> np.random.multivariate_normal(mean,
cov, 5)

Array([[ 1.55432624, -1.17972629],
       [-2.01185294, 1.96081908],
       [-2.11810813, 1.45784216],
       [-2.93207591, -0.07369322],
       [-1.37031244, -1.18408792]])
```

```
# requires the mass package

 R> install.packages('MASS')

 R> library(MASS)

R> mvrnorm(n=10, mean, cov)
[,1] [,2]
[1,] -0.8407830 -0.1882706
[2,] 0.8496822 -0.7889329
[3,] -0.1564171 0.8422177
[4,] -0.6288779 1.0618688
[5,] -0.5103879 0.1303697
[6,] 0.8413189 -0.1623758
[7,] -1.0495466 -0.4161082
[8,] -1.3236339 0.7755572
[9,] 0.2771013 1.4900494
[10,] -1.3536268 0.2338913
```